Quel modèle Ollama pour notre *détecteur de buzz et de tendances* ?

Benchmark des modèles de langage open source disponibles sur Ollama, évalués pour l'IA analyste de notre solution (sans assistant conversationnel) : lire nos calculs, trancher entre tendance, buzz et rien, et l'expliquer, sur les cinq PC de l'équipe.

IA ANALYSTE

Muse Glimmer 30B

Sur le PC 1 (48 Go), Apache 2.0. Qwen 3.6 35B (MoE) si le traitement en lot est trop lent sans GPU.

EXTRACTION EN LOT

gpt-oss 20B

Sur le PC 2 (32 Go) : JSON natif, Apache 2.0. Gemma 4 26B en alternative.

FIABILITÉ DES SOURCES

d3 9B

Modèle de décision : une probabilité par niveau de fiabilité, sans texte libre.

DÉVELOPPEMENT

Ministral 3 8B

Sur les PC 3 et 4 (16 Go). Le PC 5 (8 Go) appelle le serveur du PC 1.

Rédigé le 10/11/2026 à partir de la bibliothèque ollama.com/library, de solution.md et de config-pc.md.

Notes à confirmer par le protocole de test (partie 7).

01 · SYNTHÈSE

Ce qu'il faut retenir

Aucun modèle unique ne sert bien toutes nos briques sur toutes nos machines. La bonne réponse est un petit ensemble : un modèle fort sur le PC 1 pour expliquer les verdicts, un modèle rigoureux sur le JSON pour extraire les articles en lot, un modèle de décision pour noter la fiabilité des sources, et un modèle léger pour le développement.

- **Le facteur limitant, c'est la mémoire, pas la qualité.** Les meilleurs modèles de la bibliothèque (Muse Glimmer, Qwen 3.6, Qwen 3.8, Gemma 4 31B) pèsent de 18 à 23 Go : seul le PC 1 les charge confortablement, le PC 2 de justesse.
- **Pas de conversation, donc pas de course à la vitesse.** La solution n'a plus d'assistant conversationnel : le modèle traite les articles en lot et prépare verdicts et notes à l'avance. Un modèle plus lent mais plus rigoureux est acceptable. Sans carte graphique, les modèles MoE (3 à 4 milliards de paramètres actifs sur 26 à 35) restent le meilleur moyen de traiter beaucoup d'articles en peu de temps.
- **Le modèle doit comprendre nos mathématiques.** Il reçoit les variables calculées par la régression (pentes sur 4, 12 et 26 semaines, accélération, vitesse de montée, saisonnalité, diffusion par pays, score de durabilité et son intervalle de confiance) et doit en tirer « tendance durable », « buzz » ou « rien », sans se tromper de sens ni refaire les calculs. Nouveau critère à 15 %, appuyé par le test T4 : il favorise les modèles à mode réflexion (Muse Glimmer, Qwen 3.6, Qwen 3.8) et pénalise les petits modèles.
- **Le JSON et le raisonnement passent devant le français.** L'extraction produit des champs, pas des phrases : la rigueur du schéma pèse 20 %. Le français reste nécessaire pour l'explication du verdict et la note de décision (10 %) : Ministral 3 et Granite 4.2 le citent explicitement, Qwen 3.5 annonce 201 langues, la fiche de gpt-oss ne documente pas le multilingue.
- **Un seul serveur Ollama pour l'équipe.** Le PC 1 sert les modèles lourds à toute l'équipe sur le réseau local ; le PC 5 (8 Go) n'a pas à charger de modèle.
- **La démo garde son plan B.** config-pc.md prévoit une API pour la démo ; sans conversation, on peut aussi précalculer les analyses et les montrer sans aucun appel en direct. Le local reste l'argument de faisabilité et de RGPD (aucune donnée ne quitte la machine, aucune clé d'API dans le navigateur) et l'outil de développement gratuit.

> **Ce que ce document n'est pas.** Nous n'avons encore lancé aucun modèle sur nos PC. Les notes sont celles de l'équipe, déduites des fiches Ollama et des chiffres publiés par les éditeurs (sur leurs propres versions, pas sur les versions compressées d'Ollama). Le protocole de la partie 7 transforme ces notes en mesures sur nos propres articles.

Recommandation par machine

PC 1 48 Go · 12 cœurs

Rôle	Serveur Ollama de l'équipe, IA analyste, rejeu sur les tendances passées.
Principal	`muse-glimmer:30b` (18 Go)
Alternative	`qwen3.6:35b` (23 Go, MoE) si le lot est trop lent sur processeur seul.

PC 2 32 Go

Rôle	Extraction des articles en lot, notation de la fiabilité des sources.
Principal	`gpt-oss:20b` (14 Go) et `d3:9b` (10 Go), chargés l'un après l'autre.
Alternative	`gemma4:26b` (16 à 19 Go, MoE) si gpt-oss déçoit sur les articles en français.

PC 3 et PC 4 16 Go

Rôle	Écrire et tester les prompts, collecte, embeddings.
Principal	`ministral-3:8b` (6,0 Go) ; `d3:4b` (5,6 Go) pour tester la notation des sources.
Alternative	`qwen3.5:9b` (6,6 à 7,6 Go), `granite4.2:8b` (5,3 Go).

PC 5 8 Go

Rôle	Slides, design, vidéo ; appelle le serveur du PC 1.
Local	Aucun modèle de langage. Au plus `qwen3.5:2b` (2,7 à 3,1 Go) pour un essai isolé.
Données	Collecte Google Trends et préparation des corpus de test.

02 · BESOINS

Ce que notre solution demande au modèle

D'après solution.md, et sans l'assistant conversationnel écarté le 10/11, le modèle de langage intervient dans l'IA analyste, en quatre étapes. La régression (projection de la courbe Google Trends) reste en scikit-learn : le LLM ne fabrique aucun chiffre et ne refait aucun calcul : il lit les résultats, comprend ce qu'ils signifient, classe, explique et cite.

Lire et extraire
Presse, données de marché, requêtes associées : produits, pays, ingrédients, acteurs, en JSON.
`sorties structurées`

Juger les sources
Type de source, chiffres vérifiables, cohérence avec les autres et avec Google Trends.
`classification`

Lire les calculs
Variables de la régression, score de durabilité, intervalle : tendance, buzz ou rien, et pourquoi.
`raisonnement quantitatif`

Expliquer le verdict
Combiner le score de durabilité et les lectures, critère par critère, source par source.
`raisonnement`

Rédiger la note
Note de décision et fiche tendance générées à l'avance, sources citées, en français.
`rédaction contrainte`

Critères et poids retenus

Critère	Poids	Pourquoi il compte pour nous
Français et multilingue	10 %	Presse FR, EN, DE, ES ; explications et note de décision rédigées en français.
Sorties structurées et consignes	20 %	Extraction en JSON, respect strict du schéma et des règles (pas de chiffre inventé).
Raisonnement et explication	15 %	Expliquer un verdict critère par critère, juger la fiabilité d'une source.
Raisonnement mathématique	15 %	Comprendre pentes, accélération, saisonnalité, score de durabilité et intervalle de confiance, et en tirer tendance, buzz ou rien.
Outils et agents	10 %	Enchaîner les étapes d'analyse (agents visibles attendus par les coachs), sans dialogue.
Contexte long	10 %	Plusieurs articles et séries dans une même requête.
Tient sur nos machines	15 %	Empreinte mémoire et temps de traitement en lot sur les PC de l'équipe.
Licence claire	5 %	Règlement, article 7 : chaque outil tiers listé avec sa version et sa licence.

Comprendre la mathématique, sans la refaire

La frontière entre les trois verdicts est une affaire de chiffres : une montée brutale suivie d'une pente négative évoque un buzz ; une pente modérée, une diffusion à de nombreux pays et des requêtes associées en hausse évoquent une tendance durable ; une hausse saisonnière ou un signal trop faible ne sont « rien ». Le modèle doit donc lire correctement le sens de chaque variable, comparer un score à un seuil et tenir compte d'un intervalle de confiance large. Trois règles de conception en découlent :

- **Une fiche méthode dans le prompt.** Les définitions des variables, le sens attendu de chacune et les seuils de décision sont écrits noir sur blanc et fournis à chaque appel. Ces seuils ne sont pas encore fixés dans solution.md, notamment

pour la classe « rien » : ils sont à formaliser par Data avant les tests.

- **Le modèle ne calcule pas, il appelle.** Tout nouveau chiffre (écart au pic, ratio, moyenne) passe par une fonction Python appelée comme un outil : c'est fiable et c'est un agent qui se voit travailler.
- **Le mode réflexion activé** pour cette étape seulement, où la qualité prime sur le temps.

> **Exigence transverse : zéro chiffre inventé.** Les invariants du projet interdisent tout chiffre ou client inventé, et les coachs attendent des réponses de l'IA réelles et appuyées sur une base fournie. C'est pourquoi les sorties structurées (le modèle remplit un schéma JSON imposé par Ollama) et le respect des consignes pèsent autant que le français.

03 · MACHINES

Ce que nos PC peuvent charger

Un modèle occupe en mémoire au moins son poids de téléchargement, plus le contexte (le cache grandit avec le nombre de mots lus) et une marge pour le système, le navigateur, Docker et PostgreSQL. Nous retenons une règle prudente : **poids du modèle + 8 Go au moins de libres**.

Machine	Mémoire	Poids de modèle confortable	Ce qui passe
PC 1	48 Go	jusqu'à 30 Go environ	Tous les candidats de ce document, dont Qwen 3.6 35B (23 Go).
PC 2	32 Go	jusqu'à 19 Go environ	Gemma 4 26B, gpt-oss 20B, Mistral Small 3.2, LFM 2.5 24B. Qwen 3.6 35B trop juste.
PC 3, PC 4	16 Go	jusqu'à 7 Go environ	Ministral 3 8B, Qwen 3.5 9B, Granite 4.2 8B. gpt-oss 20B tourne en théorie avec 16 Go, mais sans marge pour le reste.
PC 5	8 Go	3 Go au plus	Modèles de 2 à 3 milliards de paramètres seulement ; mieux vaut appeler le PC 1.

> **Deux inconnues changent la vitesse, pas le choix.** config-pc.md ne dit pas encore quel système ni quelle carte graphique équipent les PC. Les 12 cœurs du PC 1 (8 performance et 4 efficacité) correspondent aussi bien à une puce Apple qu'à un processeur Intel récent. Avec une puce Apple, la mémoire est partagée avec le GPU et Ollama propose des versions MLX optimisées ; avec un PC Windows sans carte graphique, tout tourne sur le processeur et seuls les modèles MoE restent fluides. À compléter dans config-pc.md avant les tests.

04 · CANDIDATS

Les modèles retenus dans la bibliothèque

Sur plus de 200 familles listées sur ollama.com/library, nous avons écarté les modèles spécialisés en code, les modèles « cloud » qui ne tournent pas en local, les anciennes générations (Llama 3, Qwen 2.5, Mistral 7B) dépassées par leurs successeurs, et les modèles trop lourds pour 48 Go (Qwen 3.5 122B à 81 Go, gpt-oss 120B à 65 Go). Restent 13 modèles généralistes, tous avec appel d'outils.

Modèle	Tag Ollama	Poids	Contexte	Architecture	Licence	Machine
Qwen 3.6	`qwen3.6:35b`	23 Go	256K	MoE, 3B actifs	à vérifier	PC 1
Muse Glimmer	`muse-glimmer:30b`	18 Go	128K	30B (archi non précisée)	Apache 2.0	PC 1
Gemma 4 26B	`gemma4:26b`	16 à 19 Go	256K	MoE, 3,8B actifs	à vérifier	PC 1, PC 2
Qwen 3.8	`qwen3.8:27b`	18 Go	256K	27B	à vérifier	PC 1

Modèle	Tag Ollama	Poids	Contexte	Architecture	Licence	Machine
gpt-oss 20B	`gpt-oss:20b`	14 Go	128K	MoE	Apache 2.0	PC 1, PC 2
Mistral Small 3.2	`mistral-small3.2:24b`	15 Go	128K	24B dense	à vérifier	PC 1, PC 2
Nemotron 3.5 Lightning	`nemotron-3.5-lightning:30b`	19 à 25 Go	1M	MoE, 3B actifs	à vérifier	PC 1
LFM 2.5 24B	`lfm2.5:24b`	15 Go	128K	MoE, 2B actifs	à vérifier	PC 1, PC 2
Ministral 3 14B	`ministral-3:14b`	9,1 Go	256K	14B	Apache 2.0	PC 2
Ministral 3 8B	`ministral-3:8b`	6,0 Go	256K	8B	Apache 2.0	PC 3, PC 4
Qwen 3.5 9B	`qwen3.5:9b`	6,6 à 7,6 Go	256K	9B	à vérifier	PC 3, PC 4
Granite 4.2 8B	`granite4.2:8b`	5,3 Go	128K	8B dense	Apache 2.0	PC 3, PC 4
Gemma 4 E4B	`gemma4:e4b`	6,6 à 9,5 Go	128K	4,5B effectifs	à vérifier	PC 3, PC 4

Poids et contexte : fiches Ollama consultées le 10/11/2026 (version compressée par défaut). « À vérifier » : licence non indiquée sur la fiche Ollama, à relever sur Hugging Face avant de la citer dans le dossier (règlement, article 7). Qwen 3.8 a été mis à jour deux jours avant ce document : version très récente, à tester avant de s'y fier.

Deux familles complémentaires

Usage	Modèle	Poids	Ce qu'il apporte
Embedding regrouper et retrouver les articles	`qwen3-embedding:0.6b`	639 Mo	Plus de 100 langues, contexte de 32K. La version 8B (4,7 Go) a été première du classement MTEB multilingue en juin 2025.
	`bge-m3`	1,2 Go	Plus de 100 langues, recherche dense et lexicale, documents jusqu'à 8 192 mots.
	`embeddinggemma-2:270m`	moins de 400 Mo	Le plus léger ; plus de 100 langues ; vecteurs réductibles pour économiser la base.
Décision	`d3` (4B)	5,6 à 5,8 Go	Ne rédige pas : renvoie des probabilités pour des choix que l'on fournit (source faible, moyenne ou forte ; buzz ou durable). Apache 2.0. Contexte limité à 4 096 mots.

> **Pourquoi un modèle de décision nous intéresse.** Juger la fiabilité d'une source est une classification. Un modèle comme d3 donne une probabilité par niveau au lieu d'un texte, ce qui s'affiche bien dans le tableau de bord et se compare à l'annotation humaine. L'éditeur prévient que ces probabilités ne sont pas des fréquences réelles : il faut les calibrer sur nos exemples avant d'en faire un seuil.

05 · NOTATION

La grille appliquée aux 13 modèles

Notes de 1 à 5 par critère, pondérées selon la partie 2 ; total sur 5. Les notes sont des jugements de l'équipe à partir des fiches et des chiffres publiés, à remplacer par les mesures du protocole.

Modèle	Français 10 %	Sorties 20 %	Raisonnement 15 %	Raisonnement 15 %	Outils 10 %	Contexte 10 %	Tient 15 %	Licence 5 %	Total pondéré
Muse Glimmer `muse-glimmer:30b`	4	5	5	5	5	4	3	5	**4.50**
Qwen 3.6 `qwen3.6:35b`	4	4	5	5	5	5	3	3	**4.30**
Qwen 3.8 `qwen3.8:27b`	4	4	5	5	5	5	3	3	**4.30**
gpt-oss 20B `gpt-oss:20b`	3	5	4	4	4	4	4	5	**4.15**
Ministral 3 14B `ministral-3:14b`	5	4	3	3	4	5	5	5	**4.10**
Nemotron 3.5 Lightning `nemotron-3.5-lightning:30b`	3	4	4	4	5	5	4	3	**4.05**
Gemma 4 26B `gemma4:26b`	4	4	4	4	4	4	4	3	**3.95**
Qwen 3.5 9B `qwen3.5:9b`	4	4	3	3	4	5	5	3	**3.90**
Granite 4.2 8B `granite4.2:8b`	4	4	3	3	4	4	5	5	**3.90**
Ministral 3 8B `ministral-3:8b`	5	4	2	2	4	5	5	5	**3.80**
LFM 2.5 24B `lfm2.5:24b`	4	3	4	4	4	4	4	2	**3.70**
Mistral Small 3.2 `mistral-small3.2:24b`	5	4	3	2	4	4	3	3	**3.45**
Gemma 4 E4B `gemma4:e4b`	3	3	2	2	2	3	5	3	**2.90**

Lecture du classement

- **Muse Glimmer arrive en tête** : meilleurs chiffres publiés sur les consignes, le contexte long, les outils et les mathématiques (94,7 % à AIME 2026), licence Apache 2.0. Sans conversation, sa lenteur probable sur processeur seul pèse peu : l'IA analyste travaille en tâche de fond. Qwen 3.6 et Qwen 3.8 suivent de près ; Qwen 3.6 reste le recours si le lot d'articles prend trop de temps.
- **Ministral 3 14B reste le meilleur compromis à 9 Go** : français explicite, JSON natif, contexte de 256K, Apache 2.0. Sa faiblesse probable est le raisonnement mathématique, sans mode réflexion annoncé sur sa fiche : à vérifier avec T4 avant de lui confier les verdicts.
- **gpt-oss 20B remonte avec l'abandon de la conversation** : sorties structurées natives et licence Apache 2.0 en font le candidat de l'extraction en lot. Sa compréhension des articles en français reste à vérifier (T1).

- **Gemma 4 E4B ferme la marche** : les chiffres de Google montrent un net écart avec le 26B sur les tâches d'agent (42,2 % contre 68,2 % sur Tau2). À réserver aux tests rapides.

06 · CHIFFRES PUBLIÉS

Ce que disent les éditeurs

Les comparaisons directes sont rares : chaque éditeur choisit ses tests et ses concurrents. Voici les seules où plusieurs de nos candidats apparaissent dans un même tableau, sur des tests utiles à notre cas. Ce sont des chiffres sur les versions complètes, pas sur celles compressées par Ollama.

Meta compare Muse Glimmer à Gemma 4 31B et Qwen 3.6 27B (mode réflexion)

Test	Ce qu'il mesure pour nous	Muse Glimmer 30B	Gemma 4 31B	Qwen 3.6 27B
IFBench	Respect de consignes précises	**77,0**	76,0	70,8
AA-LCR	Raisonnement sur contexte long	**80,0**	68,3	73,3
MCP Atlas	Utilisation d'outils par un agent	**75,5**	54,2	62,5
DeepSearch QA	Recherche et synthèse d'informations	**74,6**	61,7	71,1
AIME 2026	Raisonnement mathématique	**94,7**	89,2	94,1
GPQA Diamond	Raisonnement scientifique	83,5	**85,7**	84,2

Source : fiche ollama.com/library/muse-glimmer. Tableau publié par Meta, éditeur de Muse Glimmer : à lire avec prudence. Qwen 3.6 27B n'est pas la version 35B MoE que nous recommandons.

Google compare ses propres tailles de Gemma 4

Test	Ce qu'il mesure pour nous	31B	26B MoE	E4B
MMMLU	Connaissances en plusieurs langues	88,4 %	86,3 %	76,6 %
AIME 2026, sans outil	Raisonnement mathématique	89,2 %	88,3 %	42,5 %
Tau2	Agent qui dialogue et utilise des outils	76,9 %	68,2 %	42,2 %
MRCR v2, 128K	Retrouver des informations dans un long contexte	66,4 %	44,1 %	25,4 %
GPQA Diamond	Raisonnement scientifique	84,3 %	82,3 %	58,6 %

Source : fiche ollama.com/library/gemma4. Le 26B MoE garde l'essentiel du 31B sur le multilingue et le raisonnement mais perd sur le contexte long : un argument pour découper les dossiers d'articles plutôt que tout envoyer d'un bloc.

> **Ce que ces chiffres ne disent pas.** AIME mesure des problèmes de concours, pas la lecture d'une régression sur des séries Google Trends. Aucun de ces tests ne porte sur la presse agroalimentaire, sur le français d'un commercial, ni sur la capacité à ne pas inventer de chiffre. C'est précisément ce que mesure notre protocole.

07 · PROTOCOLE

Comment confirmer ces notes en une demi-journée

Objectif : passer des notes de l'équipe à des mesures, sur nos propres données, avant de figer le choix dans le dossier. Cinq modèles suffisent : `muse-glimmer:30b`, `qwen3.6:35b`, `gpt-oss:20b` et `ministral-3:8b`, plus `d3:9b` pour T2.

Corpus

Une quinzaine d'articles réels et publics, en français et en anglais, sur nos cas de rejeu : Dubai Chocolate, matcha, et deux ou trois buzz retombés. Deux membres annotent à la main, pour chaque article, les produits, pays, matières premières et le niveau de fiabilité attendu. Aucune donnée personnelle.

Cinq tâches, tirées de solution.md

Tâche	Ce que l'on demande	Mesure	Seuil proposé
T1 Extraction	Remplir le schéma JSON (produits, pays, matières premières)	JSON valide ; champs justes par rapport à l'annotation	95 % de JSON valide
T2 Fiabilité	Classer la source faible, moyenne ou forte, avec la raison	Accord avec l'annotation humaine	3 articles sur 4
T3 Verdict	Expliquer un score de durabilité fourni, à partir des extraits	Chiffres absents des entrées ; sources citées qui existent	zéro chiffre inventé
T4 Calculs	Lire les variables et le score de séries réelles (Dubai Chocolate, matcha, buzz retombés, produits saisonniers) avec la fiche méthode, et rendre tendance, buzz ou rien	Accord avec la règle et avec l'issue connue ; erreurs de sens (pente lue à l'envers, saisonnalité ignorée) ; calculs refaits de tête	zéro erreur de sens
T5 Note	Rédiger en français la note de décision d'une tendance, à partir des sorties de T1 à T4	Note de 1 à 5 par deux relecteurs, à l'aveugle ; chiffres tous présents dans les entrées	moyenne de 4

Mesures de performance, renvoyées par Ollama à chaque appel

- **Vitesse de génération** : `eval_count / eval_duration`, en mots produits par seconde. Sans conversation, la vitesse sert surtout à estimer le temps de traitement d'un lot d'articles avant la démo.
- **Attente avant la réponse** : `load_duration + prompt_eval_duration`. Traiter les articles d'un même modèle à la suite pour ne le charger qu'une fois (variable `OLLAMA_KEEP_ALIVE`).
- **Mémoire réelle** : la commande `ollama ps` pendant le test.

Réglages à fixer pour tous les modèles

Température à 0 pour l'extraction et le verdict ; contexte explicite (`num_ctx`, par exemple 16 384) car la valeur par défaut d'Ollama est souvent plus courte que ce que le modèle accepte ; réflexion désactivée (`think: false`) pour T1 et T2, activée pour T3 et T4 ; schéma JSON passé dans `format` pour que la sortie soit contrainte par Ollama et non seulement demandée dans le prompt.

> **Sécurité du serveur partagé.** Pour ouvrir le PC 1 au réseau de l'équipe, Ollama s'écoute sur toutes les interfaces (`OLLAMA_HOST=0.0.0.0`). À faire uniquement sur un réseau privé : le serveur n'a pas d'authentification, et rien du projet ne doit être exposé en ligne (règlement, article 10 ; décision D5).

ANNEXE A

Script de test (Python, API locale d'Ollama)

Lance la tâche T1 sur chaque modèle et enregistre validité du JSON, vitesse et attente dans `resultats.csv`. Le corpus annoté est un fichier `corpus.jsonl` avec un article par ligne (`id`, `texte`). Les tâches T2 à T5 reprennent le même squelette avec un autre schéma et un autre prompt.

```
import csv, json, time, requests

OLLAMA = "http://localhost:11434/api/chat"   # ou l'adresse du PC 1 sur le réseau local
MODELES = ["muse-glimmer:30b", "qwen3.6:35b", "gpt-oss:20b", "ministral-3:8b"]
SCHEMA = {  # T1 : extraction structurée d'un article
  "type": "object",
  "properties": {
    "produits": {"type": "array", "items": {"type": "string"}},
    "pays": {"type": "array", "items": {"type": "string"}},
    "matieres_premieres": {"type": "array", "items": {"type": "string"}},
    "fiabilite": {"type": "string", "enum": ["faible", "moyenne", "forte"]},
    "raison_fiabilite": {"type": "string"}},
  "required": ["produits", "pays", "matieres_premieres", "fiabilite", "raison_fiabilite"]}

def appel(modele, article):
    r = requests.post(OLLAMA, timeout=600, json={
        "model": modele, "stream": False, "think": False, "format": SCHEMA,
        "options": {"temperature": 0, "num_ctx": 16384},
        "messages": [
          {"role": "system", "content": "Tu es analyste marché agroalimentaire. "
           "Réponds uniquement en JSON conforme au schéma. N'invente aucun chiffre."},
          {"role": "user", "content": article}]}).json()
    try: sortie, ok = json.loads(r["message"]["content"]), True
    except (KeyError, ValueError): sortie, ok = None, False
    tok_s = r.get("eval_count", 0) / max(r.get("eval_duration", 1), 1) * 1e9
    attente = (r.get("load_duration", 0) + r.get("prompt_eval_duration", 0)) / 1e9
    return ok, sortie, round(tok_s, 1), round(attente, 2)

with open("corpus.jsonl", encoding="utf-8") as f:      # un article annoté par ligne
    corpus = [json.loads(l) for l in f]
with open("resultats.csv", "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f); w.writerow(["modele", "article", "json_valide", "tok_s", "attente_s", "sortie"])
    for m in MODELES:
        for a in corpus:
            ok, s, v, t = appel(m, a["texte"])
            w.writerow([m, a["id"], ok, v, t, json.dumps(s, ensure_ascii=False)])
```

ANNEXE B

Limites et sources

- Les notes de la partie 5 sont des jugements, pas des mesures. Les chiffres de la partie 6 viennent des éditeurs, sur leurs versions complètes.
- La bibliothèque Ollama évolue chaque semaine (Qwen 3.8 et Qwen 3.6 ont été mis à jour dans les dix derniers jours) : relever la version exacte (`ollama show`) le jour des tests et la citer dans le dossier.
- Les licences marquées « à vérifier » doivent être relevées sur Hugging Face avant le dépôt (règlement, article 7).
- Le système, le processeur des PC 2 à 5 et la présence d'un GPU restent à compléter dans config-pc.md.

- Bibliothèque : https://ollama.com/library
- https://ollama.com/library/qwen3.6 · https://ollama.com/library/qwen3.8 · https://ollama.com/library/qwen3.5
- https://ollama.com/library/gemma4 · https://ollama.com/library/muse-glimmer · https://ollama.com/library/gpt-oss
- https://ollama.com/library/mistral-small3.2 · https://ollama.com/library/ministral-3 · https://ollama.com/library/granite4.2
- https://ollama.com/library/nemotron-3.5-lightning · https://ollama.com/library/lfm2.5
- https://ollama.com/library/qwen3-embedding · https://ollama.com/library/bge-m3 · https://ollama.com/library/embeddinggemma-2 · https://ollama.com/library/d3
- Dépôt de l'équipe : solution.md, config-pc.md, adr.md, specs.md (version du 10/11/2026).